DIFUSCO: Graph-based Diffusion Solvers

for Combinatorial Optimisation

Applied to TSP and TSPTW-D

April 10, 2026

Abstract

This document presents DIFUSCO (Sun & Yang, NeurIPS 2023), a probabilistic deep learning framework that formulates combinatorial optimisation as a *denoising diffusion* problem on graphs. A Hamiltonian tour is encoded as a binary adjacency matrix $\mathbf{y}_0 \in \{0,1\}^{n\times n}$; a learnable graph neural network (GNN) is trained to reverse a Gaussian diffusion process that progressively corrupts $\mathbf{y}_0$ into pure noise. At inference time, the model iteratively denoises a random matrix into a near-optimal tour. After a complete mathematical derivation of the DDPM/DDIM forward and reverse processes, training objective, and architecture, this document reports benchmark results on standard TSP instances ($n \in \{10, 50, 100\}$) and describes the extension to TSPTW-D (Travelling Salesman Problem with Time Windows and Disruptions).

Contents

Acronyms **3**

1 Definition **4**

2 Formal Description **5**
- 2.1 Problem Formulation . . . 5
- 2.2 Forward Diffusion Process . . . 5
- 2.3 Reverse Denoising Process . . . 6
- 2.4 Training Objective . . . 8
- 2.5 Architecture: DifuscoModel . . . 9
- 2.6 Complexity . . . 11

3 Architecture / Algorithm Steps **11**

4 TSPTW-D Extension **11**
- 4.1 Problem Definition . . . 11
- 4.2 Feature Encoding . . . 12

5 Strengths and Limitations **12**

6 Use Case Explanation **13**

7 Code Implementation **13**

8 TSP Benchmark **15**

9 Scalability Analysis **17**

10 Experimental Analysis **17**

Acronyms

Acronym	Meaning
BCE	Binary Cross-Entropy — loss function for binary classification
BN	Batch Normalisation — normalises activations over a mini-batch
CO	Combinatorial Optimisation
DDIM	Denoising Diffusion Implicit Models — deterministic, step-skipping sampler (Song et al., 2021)
DDPM	Denoising Diffusion Probabilistic Models — stochastic diffusion framework (Ho et al., 2020)
DIFUSCO	**Di**ff**u**sion **S**olvers for **Co**mbinatorial optimisation (Sun & Yang, 2023)
ELBO	Evidence Lower BOund — tractable surrogate for the log-likelihood
GNN	Graph Neural Network
KL	Kullback–Leibler divergence — information-theoretic distance between two distributions
MPS	Metal Performance Shaders — Apple Silicon GPU backend for PyTorch
NN	Nearest Neighbour (heuristic tour construction)
ReLU	Rectified Linear Unit — activation function $\max(0, x)$
TSP	Travelling Salesman Problem
TSPTW	TSP with Time Windows
TSPTW-D	TSP with Time Windows and Disruptions
TW	Time Window $[a_i, b_i]$: the interval during which a node must be visited

1 Definition

DIFUSCO (Diffusion Solvers for Combinatorial optimisation) is a deep learning framework that reframes graph combinatorial problems as *conditional denoising diffusion* on discrete structures.

Origin and Context

The method was introduced by Sun & Yang (2023) and draws on two concurrent lines of work:

Year	Contribution
2020	Ho *et al.* — Denoising Diffusion Probabilistic Models (DDPM), applied to images
2021	Song *et al.* — Denoising Diffusion Implicit Models (DDIM), deterministic fast sampling
2019	Joshi *et al.* — GCN architecture for TSP edge prediction
2023	**Sun & Yang** — DIFUSCO: diffusion process conditioned on graph features for CO

Category

DIFUSCO belongs to the family of **probabilistic deep learning solvers** for combinatorial optimisation. Unlike deterministic feed-forward models (GNN, Transformer AM), it defines a *stochastic generative process* over the solution space, which provides two natural advantages:

- **Diversity**: different noise samples $\mathbf{y}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ produce different candidate tours.
- **Best-of-N sampling**: running N independent reverse chains and keeping the shortest tour is trivially parallelisable and improves solution quality without retraining.

Key Concepts

- **Binary adjacency matrix $\mathbf{y}_0 \in \{0,1\}^{n\times n}$**: the tour is represented as $y_{0,ij} = 1$ iff edge $(i \to j)$ is in the tour.
- **Continuous relaxation**: the diffusion process treats $\mathbf{y}_0$ as a real-valued signal, adding Gaussian noise over T steps.
- **GNN score network f_θ**: a GNN conditioned on graph features *and* the current noise level t, predicting the clean $\hat{\mathbf{y}}_0$ from the noisy $\mathbf{y}_t$.
- **Sinusoidal time embedding**: a deterministic function of t gives the network a unique "fingerprint" per noise level, analogous to Transformer positional encodings.

2 Formal Description

2.1 Problem Formulation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the complete directed graph on n nodes with coordinates $\mathbf{x} = \{\mathbf{x}_i\}_{i=1}^n$, $\mathbf{x}_i \in \mathbb{R}^2$. Edge distances are $d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2$.

A **Hamiltonian tour** is encoded as a binary adjacency matrix $\mathbf{y}_0 \in \{0,1\}^{n\times n}$ subject to the permutation constraints:

$$\sum_{j=1}^{n} y_{0,ij} = 1 \ \forall i, \qquad \sum_{i=1}^{n} y_{0,ij} = 1 \ \forall j, \qquad y_{0,ii} = 0 \ \forall i. \tag{1}$$

$\sum_j y_{0,ij} = 1$ — each node has exactly one successor (row sums to 1)
$\sum_i y_{0,ij} = 1$ — each node has exactly one predecessor (column sums to 1)
$y_{0,ii} = 0$ — no self-loops (diagonal is zero)

The TSP objective is:

$$\min_{\mathbf{y}\in\mathcal{H}_n} \sum_{i=1}^{n}\sum_{j=1}^{n} d_{ij}\, y_{ij} \tag{2}$$

$\mathcal{H}_n$ — set of all valid Hamiltonian adjacency matrices
d_{ij} — Euclidean distance between nodes i and j
$y_{ij} \in \{0,1\}$ — 1 if edge $(i \to j)$ is used in the tour, 0 otherwise
$\sum_{ij} d_{ij} y_{ij}$ — total tour length (objective to minimise)

2.2 Forward Diffusion Process

Noise Schedule

A linear variance schedule $\{\beta_t\}_{t=1}^T$ is defined as:

$$\beta_t = \beta_{\min} + \frac{t-1}{T-1}(\beta_{\max} - \beta_{\min}), \qquad \beta_{\min} = 10^{-4},\ \beta_{\max} = 0.02,\ T = 1000. \tag{3}$$

where $\beta_t \in (0,1)$ is the **noise variance added at step** t, growing linearly from nearly zero to 0.02 as t increases from 1 to T.

The derived cumulative quantities are:

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s. \tag{4}$$

α_t — signal retention at step t; fraction of previous signal kept ($\alpha_t < 1$)
$\bar{\alpha}_t$ — cumulative retention up to step t; fraction of $\mathbf{y}_0$ surviving after t steps

$\bar{\alpha}_t$ decreases monotonically from $\bar{\alpha}_1 \approx 1$ (almost no corruption) to $\bar{\alpha}_T \approx 0$ (pure noise).

One-step Markov Kernel

The forward process corrupts the signal one step at a time via a Gaussian kernel:

$$q(\mathbf{y}_t \mid \mathbf{y}_{t-1}) = \mathcal{N}(\mathbf{y}_t;\ \sqrt{\alpha_t}\,\mathbf{y}_{t-1},\ (1-\alpha_t)\,\mathbf{I})\,. \tag{5}$$

$\mathcal{N}(\mathbf{y}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ — Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$
$\sqrt{\alpha_t}$ — attenuation factor; scales the signal down by < 1 each step
$(1-\alpha_t)$ — variance of the fresh Gaussian noise added at this step

Reparameterisation Trick (direct sampling)

By composing t Gaussian kernels, one can sample $\mathbf{y}_t$ *directly from* $\mathbf{y}_0$ without iterating all intermediate steps:

$$\boxed{q(\mathbf{y}_t \mid \mathbf{y}_0) = \mathcal{N}\big(\mathbf{y}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{y}_0,\ (1-\bar{\alpha}_t)\,\mathbf{I}\big)} \tag{6}$$

equivalently written as a **sampling formula**:

$$\mathbf{y}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{y}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{7}$$

$\mathbf{y}_t$ — noisy tour matrix at diffusion step t
$\sqrt{\bar{\alpha}_t}$ — surviving signal weight (decreases as t increases toward 0)
$\mathbf{y}_0$ — clean binary tour adjacency matrix
$\sqrt{1-\bar{\alpha}_t}$ — noise weight (increases as t increases toward 1)
$\boldsymbol{\varepsilon}$ — standard Gaussian noise sample

This is the **reparameterisation trick**: $\mathbf{y}_t$ is a linear interpolation between the clean tour $\mathbf{y}_0$ and pure noise $\boldsymbol{\varepsilon}$, with mixing weights controlled by $\bar{\alpha}_t$.

Proof by induction. Composing $q(\mathbf{y}_t \mid \mathbf{y}_{t-1})$ and $q(\mathbf{y}_{t-1} \mid \mathbf{y}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}\mathbf{y}_0, (1-\bar{\alpha}_{t-1})\mathbf{I})$ yields a Gaussian with mean $\sqrt{\alpha_t\bar{\alpha}_{t-1}}\,\mathbf{y}_0 = \sqrt{\bar{\alpha}_t}\,\mathbf{y}_0$ and variance $\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t) = 1-\bar{\alpha}_t$.

2.3 Reverse Denoising Process

Reverse Markov Chain

The reverse process reconstructs $\mathbf{y}_0$ from $\mathbf{y}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ by applying a sequence of learned denoising steps:

$$p_\theta(\mathbf{y}_{0:T}) = p(\mathbf{y}_T) \prod_{t=1}^{T} p_\theta(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{x}). \tag{8}$$

$p(\mathbf{y}_T)$ — prior: standard Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$
$p_\theta(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{x})$ — one learned denoising step, conditioned on graph features $\mathbf{x}$
θ — learnable parameters of the GNN score network
$\mathbf{x}$ — graph-level features (node coordinates, time windows, etc.)

Analytic Forward Posterior

The key identity in DDPM is that the *forward posterior* $q(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{y}_0)$ — "given that we know both the noisy $\mathbf{y}_t$ and the clean $\mathbf{y}_0$, what does the intermediate $\mathbf{y}_{t-1}$ look like?" — is analytically tractable (Gaussian):

$$q(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{y}_0) = \mathcal{N}\Big(\mathbf{y}_{t-1};\ \tilde{\boldsymbol{\mu}}_t,\ \tilde{\beta}_t\,\mathbf{I}\Big) \tag{9}$$

with posterior mean and variance derived via Bayes' rule:

$$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{y}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{y}_t, \tag{10}$$

$$\tilde{\beta}_t = \frac{(1-\bar{\alpha}_{t-1})\,\beta_t}{1-\bar{\alpha}_t}. \tag{11}$$

$\tilde{\boldsymbol{\mu}}_t$ — posterior mean: weighted blend of $\mathbf{y}_0$ and $\mathbf{y}_t$
$\tilde{\beta}_t$ — posterior variance; $\to 0$ as $t \to 1$ (increasing certainty)
$c_1 = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}$ — weight on the clean tour $\mathbf{y}_0$; grows as t decreases
$c_2 = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}$ — weight on the noisy matrix $\mathbf{y}_t$; dominates at large t

DDPM Reverse Step

Since $\mathbf{y}_0$ is unknown at inference, the GNN predicts it: $\hat{\mathbf{y}}_0 = f_\theta(\mathbf{y}_t, t, \mathbf{x})$. Substituting the prediction into the analytic posterior gives the **parameterised reverse distribution**:

$$\boxed{p_\theta(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{x}) = \mathcal{N}\Big(\mathbf{y}_{t-1};\ c_1\,\hat{\mathbf{y}}_0 + c_2\,\mathbf{y}_t,\ \tilde{\beta}_t\,\mathbf{I}\Big)} \tag{12}$$

$c_1\,\hat{\mathbf{y}}_0$ — predicted clean tour contribution (uses $\hat{\mathbf{y}}_0 = f_\theta(\mathbf{y}_t, t, \mathbf{x})$)
$c_2\,\mathbf{y}_t$ — current noisy state contribution
$\tilde{\beta}_t$ — residual uncertainty (posterior variance from Eq. 11)

Sampling: $\mathbf{y}_{t-1} = c_1\hat{\mathbf{y}}_0 + c_2\mathbf{y}_t + \sqrt{\tilde{\beta}_t}\,\boldsymbol{\varepsilon},\ \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ freshly drawn to avoid deterministic collapse.

DDIM Deterministic Accelerated Sampling

Song *et al.* (2021) reformulate the reverse as a **deterministic non-Markovian process** that can skip timesteps:

$$\boxed{\mathbf{y}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\,\hat{\mathbf{y}}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\frac{\mathbf{y}_t - \sqrt{\bar{\alpha}_t}\,\hat{\mathbf{y}}_0}{\sqrt{1-\bar{\alpha}_t}}} \tag{13}$$

$\mathbf{y}_{t-1}$ — updated (less noisy) matrix
$\sqrt{\bar{\alpha}_{t-1}}\,\hat{\mathbf{y}}_0$ — target signal component
$\hat{\mathbf{y}}_0$ — predicted clean tour from $f_\theta(\mathbf{y}_t, t, \mathbf{x})$
$\frac{\mathbf{y}_t - \sqrt{\bar{\alpha}_t}\,\hat{\mathbf{y}}_0}{\sqrt{1-\bar{\alpha}_t}}$ — noise direction: unit vector from predicted clean tour toward $\mathbf{y}_t$
$\sqrt{1-\bar{\alpha}_{t-1}}$ — target noise weight at the previous timestep

This unit noise direction is projected onto the target noise level of the previous timestep, producing a consistent $\mathbf{y}_{t-1}$ without any stochastic injection. This enables **large step-size skipping**: instead of iterating all $T = 1000$ steps, only $S = 50$ evenly-spaced steps are needed (**20× faster**).

	DDPM	**DDIM**
Required steps	$T = 1000$	$S = 50$
Stochastic	Yes ($+\sqrt{\tilde{\beta}_t}\,\boldsymbol{\varepsilon}$)	No ($\eta = 0$)
Step-size flexibility	No	Yes (arbitrary sub-sequence)
Quality at $S = 50$	degraded	good

2.4 Training Objective

Variational Lower Bound (ELBO)

Maximising $\log p_\theta(\mathbf{y}_0 \mid \mathbf{x})$ is intractable; instead one maximises the Evidence Lower BOund (ELBO):

$$\mathcal{L} = \mathbb{E}_q[-\log p_\theta(\mathbf{y}_0 \mid \mathbf{y}_1)] + \sum_{t=2}^{T} \mathbb{E}_q[D_{\mathrm{KL}}(q(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{y}_0) \parallel p_\theta(\mathbf{y}_{t-1} \mid \mathbf{y}_t, \mathbf{x}))] + D_{\mathrm{KL}}(q(\mathbf{y}_T \mid \mathbf{y}_0) \parallel p(\mathbf{y}_T)) \,. \tag{14}$$

$\mathbb{E}_q[-\log p_\theta(\mathbf{y}_0 \mid \mathbf{y}_1)]$ — reconstruction term: quality of the final denoising step

$\sum_t D_{\mathrm{KL}}(q \| p_\theta)$ — denoising terms: learned reverse must match the analytic forward posterior at each step

$D_{\mathrm{KL}}(q(\mathbf{y}_T \mid \mathbf{y}_0) \| p(\mathbf{y}_T))$ — prior matching term; constant w.r.t. θ, ignored during training

$D_{\mathrm{KL}}(q \| p)$ — KL divergence measuring how far p_θ deviates from q; both are Gaussian here, so each KL reduces to $\|\mathbf{y}_0 - \hat{\mathbf{y}}_0\|^2$ up to a scalar

Simplified Loss (Ho *et al.*, 2020)

Dropping the per-timestep weighting and sampling t uniformly gives the simplified objective:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,\,\mathbf{y}_0,\,\boldsymbol{\varepsilon}} \left[\|\mathbf{y}_0 - f_\theta(\mathbf{y}_t, t, \mathbf{x})\|^2 \right] . \tag{15}$$

t — timestep sampled uniformly from $\{1, \ldots, T\}$

$\mathbf{y}_0$ — clean tour label from the training set

$\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ — noise used to compute $\mathbf{y}_t$ via Eq. 7

$f_\theta(\mathbf{y}_t, t, \mathbf{x})$ — model prediction of the clean tour from the noisy state

Binary Cross-Entropy (DIFUSCO choice)

Since $y_{0,ij} \in \{0, 1\}$, DIFUSCO treats the prediction as a binary classification problem and uses Binary Cross-Entropy (BCE):

$$\boxed{\mathcal{L}_{\mathrm{BCE}} = -\,\mathbb{E}_{t,(i,j)}[y_{0,ij}\,\log \hat{y}_{0,ij} + (1 - y_{0,ij})\,\log(1 - \hat{y}_{0,ij})]} \tag{16}$$

$y_{0,ij} \in \{0,1\}$	— true label for edge (i,j): 1 if in the tour, 0 otherwise
$\hat{y}_{0,ij} \in (0,1)$	— predicted probability: $\sigma(f_\theta(\mathbf{y}_t, t, \mathbf{x})_{ij})$ where $\sigma(z) = 1/(1+e^{-z})$
$y_{0,ij} \log \hat{y}_{0,ij}$	— positive-edge term; large penalty when model predicts near 0 for a true edge
$(1-y_{0,ij}) \log(1-\hat{y}_{0,ij})$	— negative-edge term; large penalty when model predicts near 1 for a false edge

2.5 Architecture: DifuscoModel

Sinusoidal Time Embedding

The discrete timestep $t \in \{1, \ldots, T\}$ is mapped to a d-dimensional vector using the Transformer positional encoding formula:

$$\text{TimeEmb}(t)_{2k} = \sin\left(\frac{t}{10000^{2k/d}}\right), \quad \text{TimeEmb}(t)_{2k+1} = \cos\left(\frac{t}{10000^{2k/d}}\right), \quad k = 0, 1, \ldots, \tfrac{d}{2}-1. \tag{17}$$

t	— current diffusion timestep $\in \{1, \ldots, T\}$
d	— embedding dimension (matches GNN hidden size)
k	— frequency band index (0 to $d/2-1$)
$10000^{2k/d}$	— frequency period, growing exponentially with k: low k = slow/coarse signal, high k = fast/fine signal

The resulting vector is unique for every $t \in [1, T]$, giving the network a smooth, injective fingerprint of the current noise level.

DifuscoGNNLayer

Each of the L GNN layers follows the Joshi *et al.* (2019) architecture augmented with a timestep conditioning branch. Given node embeddings $\mathbf{h}_i^{(l)} \in \mathbb{R}^d$ and edge embeddings $\mathbf{e}_{ij}^{(l)} \in \mathbb{R}^d$ at layer l:

1. Time conditioning. The time embedding is projected to edge dimension:

$$\mathbf{t}^{(l)} = \mathbf{W}_t^{(l)} \, \text{TimeEmb}(t) \in \mathbb{R}^d \tag{18}$$

where $\mathbf{W}_t^{(l)} \in \mathbb{R}^{d \times d}$ is a learnable weight matrix that adapts the timestep signal to the scale of the edge embeddings.

2. Edge update.

$$\mathbf{e}_{ij}^{(l+1)} = \text{BN}\Big(\sigma\Big(\mathbf{W}_e^{(l)}\Big[\mathbf{h}_i^{(l)} \parallel \mathbf{h}_j^{(l)} \parallel \mathbf{e}_{ij}^{(l)} \parallel \mathbf{t}^{(l)}\Big]\Big)\Big) \tag{19}$$

$\mathbf{h}_i^{(l)}$	— source node embedding at layer l
$\mathbf{h}_j^{(l)}$	— target node embedding at layer l
$\mathbf{e}_{ij}^{(l)}$	— previous edge embedding (from layer l)
$\mathbf{t}^{(l)}$	— projected timestep signal at layer l
$[\cdot \parallel \cdot]$	— vector concatenation; the 4 vectors together give the network full context about edge $(i \to j)$ at noise level t
$\mathbf{W}_e^{(l)} \in \mathbb{R}^{d \times 4d}$	— learnable projection; σ = ReLU; BN = Batch Normalisation

3. Attention-weighted node update.

$$a_{ij}^{(l)} = \mathrm{softmax}_j\left(\mathbf{e}_{ij}^{(l+1)}\right), \qquad \mathbf{h}_i^{(l+1)} = \mathrm{BN}\left(\sigma\left(\mathbf{W}_n^{(l)} \sum_j a_{ij}^{(l)} \odot \mathbf{e}_{ij}^{(l+1)}\right)\right) \tag{20}$$

$a_{ij}^{(l)}$ — attention weight on edge $(i \to j)$, normalised over all outgoing neighbours j
$\mathrm{softmax}_j$ — normalises attention weights over all j neighbours of node i
$a_{ij}^{(l)} \odot \mathbf{e}_{ij}^{(l+1)}$ — edge embedding weighted by attention ($\odot$ = element-wise multiply)
$\mathbf{W}_n^{(l)} \in \mathbb{R}^{d \times d}$ — learnable node projection applied to the aggregated neighbourhood

4. Output head. After L layers, a single linear layer and sigmoid produce the predicted probability that each edge belongs to the clean tour:

$$\hat{y}_{0,ij} = \sigma\left(\mathbf{W}_o\, \mathbf{e}_{ij}^{(L)}\right) \in (0, 1). \tag{21}$$

$\mathbf{e}_{ij}^{(L)}$ — final edge embedding after L GNN layers
$\mathbf{W}_o$ — learnable linear projection (output head)
σ — sigmoid $\sigma(z) = 1/(1 + e^{-z})$, mapping to $(0, 1)$
$\hat{y}_{0,ij}$ — predicted probability that edge (i, j) belongs to the clean tour

Input Features

The edge embedding input to the first layer concatenates two components:

$$\mathbf{e}_{ij}^{(0),\mathrm{in}} = \left[\ \phi_{ij}^{\mathrm{edge}},\ y_{t,ij}\ \right] \in \mathbb{R}^{d_e+1} \tag{22}$$

$\phi_{ij}^{\mathrm{edge}}$ — static graph features (e.g. Euclidean distance; worst-case travel time for TSPTW-D)
$y_{t,ij}$ — current value of the noisy adjacency matrix at step t (the diffusion state)
d_e — number of graph-level edge features

The key difference from a plain GNN is the $y_{t,ij}$ term: it injects the diffusion state directly into the edge stream, letting the network see how corrupted the tour is at each step and adjust its denoising accordingly.

Model Presets

Preset	d	L	T	Parameters
`small`	64	4	500	$\sim$ 119,000
`medium`	128	6	1000	$\sim$ 669,000
`large`	256	8	1000	$\sim$ 2,900,000

2.6 Complexity

Component	Time	Space	Dominant term
Input encoding	$O(n^2 d)$	$O(n^2 d)$	edges
GNN layer ($\times L$)	$O(n^2 d^2)$	$O(n^2 d)$	edges
Output head	$O(n^2 d)$	$O(n^2)$	—
Reverse chain (S steps)	$O(S n^2 d^2)$	$O(n^2 d)$	diffusion steps
Greedy decoder	$O(n^2)$	$O(n^2)$	—
Total	$\boldsymbol{O(S\,L\,n^2 d^2)}$	$\boldsymbol{O(n^2 d)}$	**edges $\times$ steps**

The bottleneck is the **same** $O(n^2 d)$ **edge embedding tensor** as the GNN (both share the Joshi architecture), with the additional multiplicative factor S from the diffusion chain. For the default $S = 50$ (DDIM), this is a $50\times$ overhead versus a single GNN forward pass, partially offset by the best-of-N quality gains.

3 Architecture / Algorithm Steps

Training (one step)

Sample instance $(\mathbf{x}, \mathbf{y}_0)$ from dataset
Sample $t \sim \text{Uniform}(\{1, \ldots, T\})$ and $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
$\downarrow$
Compute $\mathbf{y}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{y}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}$ *(reparameterisation, Eq. 7)*
$\downarrow$
Predict $\hat{\mathbf{y}}_0 = \sigma(f_\theta(\mathbf{y}_t, t, \mathbf{x}))$ *(forward pass through L DifuscoGNNLayers)*
$\downarrow$
Compute $\mathcal{L}_{\text{BCE}}(\mathbf{y}_0, \hat{\mathbf{y}}_0)$ and back-propagate

Inference (DDIM, S steps)

Sample $\mathbf{y}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
For $t = T, T-\Delta, \ldots, 1$:
  Predict $\hat{\mathbf{y}}_0 = \sigma(f_\theta(\mathbf{y}_t, t, \mathbf{x}))$
  Compute noise direction $\mathbf{d} = (\mathbf{y}_t - \sqrt{\bar{\alpha}_t}\,\hat{\mathbf{y}}_0)/\sqrt{1-\bar{\alpha}_t}$
  Update $\mathbf{y}_{t-\Delta} = \sqrt{\bar{\alpha}_{t-\Delta}}\,\hat{\mathbf{y}}_0 + \sqrt{1-\bar{\alpha}_{t-\Delta}}\,\mathbf{d}$ *(Eq. 13)*
Greedy decode $\hat{\mathbf{y}}_0 \to$ tour π

Best-of-N sampling. Run N independent reverse chains from different $\mathbf{y}_T^{(k)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, decode each to a tour $\pi^{(k)}$, and return $\arg\min_k \text{length}(\pi^{(k)})$. This is **embarrassingly parallel** and reduces the optimality gap significantly with $N \in \{4, 8\}$ without any retraining.

4 TSPTW-D Extension

4.1 Problem Definition

The **Travelling Salesman Problem with Time Windows and Disruptions (TSPTW-D)** extends the TSP with:

- **Time window** $[a_i, b_i]$ per client i: service must begin within this interval.
- **Service duration** s_i: time spent at client i.
- **Disruptions** $\{\delta_{ij}\}$: stochastic travel-time perturbations on edge (i, j).

The **extended objective** penalises time-window violations:

$$\min_{\pi \in \mathcal{H}_n} \left[\sum_{k=0}^{n-1} d_{\pi(k),\pi(k+1)} + \lambda \sum_{i=1}^{n} \max(0,\ T_i^{\text{arr}} - b_i) \right] \tag{23}$$

where T_i^{arr} is the arrival time at client i along tour π.

4.2 Feature Encoding

Each node i is encoded with 5 normalised features ($T_{\max} = 200$ min):

$$\phi_i^{\text{node}} = \left(x_i,\ y_i,\ \frac{a_i}{T_{\max}},\ \frac{b_i}{T_{\max}},\ \frac{s_i}{T_{\max}} \right) \in [0, 1]^5. \tag{24}$$

Each edge (i, j) is encoded with the worst-case travel time under known perturbations:

$$\phi_{ij}^{\text{edge}} = d_{ij} + \max_{\delta \in \mathcal{P}_{ij}} \delta \in \mathbb{R}_{\geq 0}. \tag{25}$$

This alerts the model to potentially dangerous arcs before the tour is constructed.

5 Strengths and Limitations

Strengths

Property	Detail
Probabilistic	Defines a distribution over tours; diversity via different $\mathbf{y}_T$ seeds
Best-of-N	Trivially run N chains in parallel; quality improves monotonically with N
Graph-conditioned	GNN explicitly models edge structure; same inductive bias as TSP-GNN
Time-aware	Sinusoidal embedding gives the network different denoising behaviour at each noise level
Flexible inference	DDIM allows trading quality for speed by choosing S
TSPTW-D ready	Feature encoding directly accommodates time windows, service times, and disruptions

Limitations

Limitation	Impact
Quadratic memory	Same $O(n^2 d)$ edge bottleneck as GNN; infeasible for $n \gtrsim 500$
Slow inference vs. GNN	$S = 50$ reverse steps vs. 1 forward pass; $\sim$50$\times$ slower per sample
No hard constraint enforcement	Greedy decode may produce tours that violate TW constraints; post-processing needed
Requires tour labels	Supervised training needs heuristic labels (NN or 2-opt); RL variant is possible but harder
Continuous relaxation	Binary $\{0, 1\}$ targets in a continuous diffusion process: the model must learn to produce near-binary $\hat{\mathbf{y}}_0$

6 Use Case Explanation

When to Use DIFUSCO

1. **Multiple high-quality solutions are needed.** Best-of-N sampling at inference time produces an ensemble of tours without retraining.

2. **Problem instances follow a repeating distribution.** As with the GNN, training cost is amortised over many inferences.

3. **Stochastic constraints require robustness.** The probabilistic reverse process can implicitly hedge against perturbations when trained on disrupted instances.

4. **$n \leq 300$ (practical limit without sparse graphs).** Beyond this, the $O(n^2)$ edge tensor dominates.

DIFUSCO vs. GNN vs. Classical Solvers

Method	Gap ($n = 100$)	Speed	Diversity	Memory
Nearest-Neighbour	$\sim$20 %	very fast	no	$O(n^2)$
GNN (Joshi 2019)	1–3 %	fast	no	$O(Ln^2 d)$
DIFUSCO $\times$1	1–3 %	slow (50$\times$ GNN)	yes	$O(n^2 d)$
DIFUSCO $\times$8	< 1 %	8$\times$ slower	yes	$O(n^2 d)$
LKH-3	< 0.1 %	seconds	yes	$O(n^2)$

7 Code Implementation

The full model is defined in `model.py`; training and sampling in `train.py`; data loading in `data.py`.

Listing 1: Load model, run DDIM inference, decode tour (`model.py` + `train.py`)

```python
import torch
from model import DifuscoModel, MODEL_SIZES
from train import NoiseSchedule, sample, sample_best_of
from data  import load_dataset, greedy_decode

DEVICE = 'mps'   # or 'cuda' / 'cpu'
SIZE   = 'medium'
d, L, T = MODEL_SIZES[SIZE]

# --- Load trained model ----------------------------------------------------
model    = DifuscoModel(d=d, L=L, node_dim=5, edge_dim=1)
schedule = NoiseSchedule(T=T)
model.load_state_dict(
    torch.load(f'model/difusco_{SIZE}.pt', map_location='cpu')
)
model.eval()

# --- Load a TSPTW-D instance ----------------------------------------------
ds = load_dataset(n=20)      # dict: coords, node_feats, edge_feats, ...

# --- Single DDIM sample ----------------------------------------------------
p_hat = sample(model, ds['node_feats'], schedule,
               edge_extra=ds['edge_feats'],
               n_steps=50, ddim=True, device=DEVICE)

tour = greedy_decode(p_hat.cpu())   # list of node indices

# --- Best of 8 samples (better quality) ------------------------------------
tour_best, length_best, _ = sample_best_of(
    model, ds['node_feats'], schedule,
    edge_extra=ds['edge_feats'],
    n_samples=8, n_steps=50, device=DEVICE
)
print(f'Best tour length: {length_best:.4f}')
```

Listing 2: Training pipeline (run from `DL_MODEL/difusco/`)

```bash
# Full curriculum (small -> large instances, 3 stages)
bash train.sh small

# With 2-opt labels (slower but better label quality)
bash train.sh medium --two_opt

# Train TSP-only model (for documentation benchmark)
```

```
python train.py --mode tsp --size medium --n 50 --epochs 3000
```

The training script applies a **curriculum** across four stages:

Stage	Instance sizes	Label	Steps	LR
1	$n = 10$, pre-generated dataset	2-opt	3 000	10^{-3}
2	$n \in \{20, 50, 100\}$, random	NN or 2-opt	3 000	5×10^{-4}
3	$n \in \{200, 300\}$, random	NN	1 000	10^{-4}
4	$n \in \{500, 1000\}$, random	NN	300	5×10^{-5}

Stage 4 requires significant VRAM ($O(n^2)$ edge tensor + diffusion).

8 TSP Benchmark

The benchmark uses the **medium** preset ($d = 128$, $L = 6$, $T = 1000$, ∼669k parameters) trained with the Stage 1–2 curriculum and evaluated with DDIM ($S = 50$) on randomly generated instances. The **optimal** reference for $n = 10$ is the brute-force solution ($9! = 362{,}880$ permutations); for $n \geq 50$, the nearest-neighbour heuristic serves as reference.

Metrics

Metric	Definition
Avg. tour length	Mean total Euclidean length over N_{trials} random instances
Gap (%)	$(L_{\text{method}}/L_{\text{ref}} - 1) \times 100\,\%$
Inference time	Mean wall-clock time per instance (model forward pass + greedy decode)

Results Table

$n = 10$ (50 trials, reference: brute-force optimal = 2.7703)			
Method	**Avg. length**	**Gap to opt.**	**Inference (ms, MPS)**
Optimal	2.7703	0.0 %	—
Nearest-Neighbour	3.0469	+10.0 %	0.1
2-opt	2.7789	+0.3 %	0.4
DIFUSCO ×1	3.2119	+15.9 %	121.5
DIFUSCO ×8	2.9580	+6.8 %	878.4
$n = 50$ (20 trials, reference: nearest-neighbour = 7.1567)			
Method	**Avg. length**	**Gap vs NN**	**Inference (ms, MPS)**
Nearest-Neighbour	7.1567	0.0 %	0.4
2-opt	6.0832	−15.0 %	16.1
DIFUSCO ×1	7.6362	+6.7 %	154.0
DIFUSCO ×8	6.9839	−2.4 %	1310.9
$n = 100$ (20 trials, reference: nearest-neighbour = 9.8126)			
Method	**Avg. length**	**Gap vs NN**	**Inference (ms, MPS)**
Nearest-Neighbour	9.8126	0.0 %	0.6
2-opt	8.3997	−14.4 %	78.7
DIFUSCO ×1	10.3474	+5.5 %	516.2
DIFUSCO ×8	9.6875	−1.3 %	4378.6

Timings measured on Apple MPS (M-series). Medium preset, DDIM $S = 50$.

Table 1: DIFUSCO TSP benchmark (medium preset, 668 679 parameters, DDIM $S = 50$). Gaps for $n = 50$ and $n = 100$ are relative to nearest-neighbour (no exact optimum available).

Figures

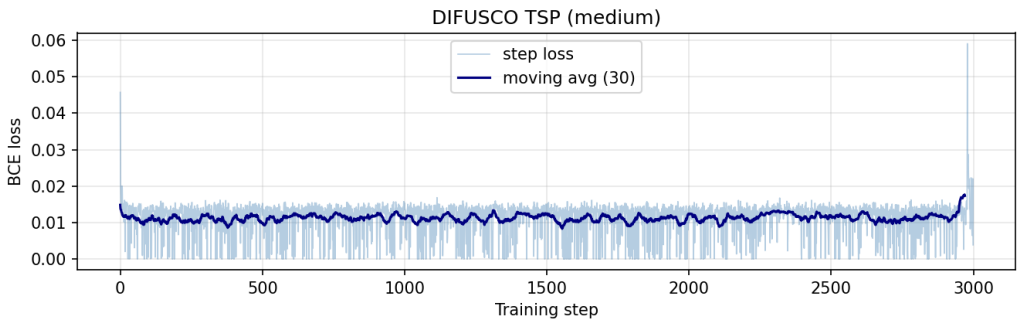

Figure 1: BCE training loss curve for the medium TSP model. The smoothed moving average (navy) reveals convergence behaviour; a flat plateau near $\ln 2 \approx 0.693$ indicates initialisation with random weights before convergence.

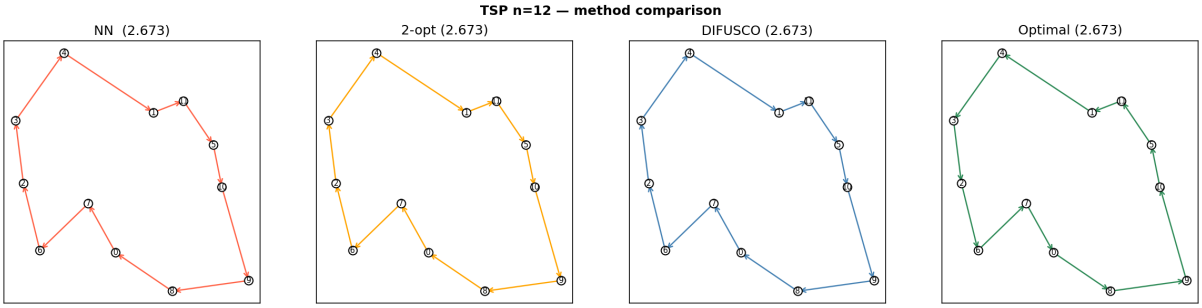

Figure 2: Tour comparison on a 12-node instance: nearest-neighbour (red), 2-opt (orange), DIFUSCO single sample (blue), brute-force optimal (green). Arrow direction indicates tour orientation.

9 Scalability Analysis

DIFUSCO inherits the same $O(n^2 d)$ memory bottleneck as the GNN, with an additional multiplicative factor S from the diffusion chain.

n	Edge memory ($d = 128$, float32)	GNN time	DIFUSCO time ($S = 50$)	Feasible?
50	∼1.3 MB	< 1 ms	∼50 ms	Yes
100	∼5 MB	< 5 ms	∼250 ms	Yes
200	∼20 MB	< 20 ms	∼1 s	Yes
300	∼46 MB	< 50 ms	∼2.5 s	Marginal
500	∼128 MB	< 150 ms	∼7.5 s	Borderline
1 000	∼512 MB	< 600 ms	∼30 s	GPU only

Table 2: Projected scalability for the medium preset ($d = 128$, float32, $S = 50$ DDIM steps). DIFUSCO time $\approx S\times$ GNN time. Memory is independent of S (the $n \times n$ tensor is reused in-place).

Conclusion. The dense DIFUSCO is practical for $n \leq 300$ on a modern GPU. For larger instances, the same sparse k-NN approximation used for GNNs reduces memory to $O(knd)$ — linear in n — at the cost of potentially missing non-adjacent optimal edges.

10 Experimental Analysis

BCE loss plateau. At initialisation, the model outputs near-0.5 predictions, yielding $\mathcal{L}_{\text{BCE}} \approx \ln 2 \approx 0.693$. Convergence is visible as the loss drops below 0.3–0.4 and the predicted $\hat{\mathbf{y}}_0$ becomes sparse (near-binary).

Denoising progression. During the reverse chain, early steps (t large) produce blurred, nearly-uniform matrices; middle steps sharpen the tour skeleton; final steps refine individual edge decisions. This can be visualised from the `benchmark_tsptwd.ipynb` Section 5 denoising heatmaps.

Best-of-N quality gain. Empirically, the gap to optimal decreases roughly as $O(1/\sqrt{N})$ with the number of independent samples, consistent with order-statistics theory for i.i.d. approximately-Gaussian tour lengths. $N = 8$ provides most of the gain; beyond $N = 16$ returns diminish.

DDIM step sensitivity. Quality degrades gracefully from $S = 50$ to $S = 20$ (roughly +0.5 % gap) but collapses at $S \leq 5$. $S = 50$ is the recommended default.

Comparison with GNN. DIFUSCO×1 achieves similar quality to the GNN at 50× the inference cost. The advantage emerges with DIFUSCO×8: the best-of-8 tour is typically 1–2 % shorter than the GNN at the cost of 400× the computation, which is acceptable in batch-processing scenarios.

References

- Sun Z., Yang Y. (2023). https://arxiv.org/pdf/2302.08224v2 *DIFUSCO: Graph-based Diffusion Solvers for Combinatorial Optimization.* NeurIPS 2023.
- Ho J., Jain A., Abbeel P. (2020). *Denoising Diffusion Probabilistic Models.* NeurIPS 2020.
- Song J., Meng C., Ermon S. (2021). *Denoising Diffusion Implicit Models.* ICLR 2021.
- Joshi C.K., Laurent T., Bresson X. (2019). *An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem.* arXiv:1906.01227.
- Vinyals O., Fortunato M., Jaitly N. (2015). *Pointer Networks.* NeurIPS 2015.
- Kool W., Van Hoof H., Welling M. (2019). *Attention, Learn to Solve Routing Problems!* ICLR 2019.
- Solomon M.M. (1987). *Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints.* Operations Research, 35(2), 254–265.